

I PROMPT TEMPLATES

This appendix reports source-aligned prompt excerpts used by the continuous decision-driven structural evolution framework. We preserve the original prompt sections, instruction styles, and output schemas, while replacing repository-specific runtime payloads with placeholders.

A. Requirement Extraction

The requirement extraction stage turns a README or project document into a capability-level requirement set, filters out setup-only material, and merges semantically redundant requirements before dependency construction.

Requirement Extraction Prompt

You are a Senior Technical Product Manager and Business Analyst. Your task is to analyze the following project documentation and extract all functional requirements.

CLASSIFICATION

Requirements: The system must include a set of functionalities that are essential for performing its core tasks, while also optimizing performance, enhancing user experience, and improving overall efficiency. These functionalities work together to ensure the system operates effectively, reliably, and meets its intended objectives.

INSTRUCTIONS

1. Infer requirements from project documentation, dependencies, features, and implementation details. EXCLUDE all environmental, setup, and general configuration settings.
2. Strictly output a single, valid JSON object that adheres precisely to the required JSON schema.
3. Based on the previously extracted requirements, attempt to identify additional requirements that were not captured in previous rounds.

HIERARCHICAL REQUIREMENT STRUCTURE

- Extract requirements at an appropriate level: major capabilities should be separate requirements.
- For each major capability, if it has multiple related operations or features, include them comprehensively in the description field.
- Do not split low-level operations into separate requirements.
- The description should capture implementation details and sub-features of that capability.

INPUT PLACEHOLDERS

```
[Project Documentation]: {documentation_content}
PREVIOUSLY EXTRACTED REQUIREMENTS:
{requirements_list}
```

REQUIRED JSON SCHEMA

```
{"project_summary": "A concise summary of the
project's purpose.",
  "requirements": [
    {"name": "string", "description": "string"}
  ]
}
```

Requirement Deduplication Prompt

You are a Senior Product Analyst specializing in requirements engineering and deduplication.

TASK

Analyze the provided list of requirements and identify items that are functionally redundant or significantly overlapping. Merge them into a single, comprehensive requirement.

INPUT DATA (JSON)

```
{requirements_payload}
```

MERGING CRITERIA

1. Merge when two or more requirements describe the same user goal or system behavior using different wording.
2. Merge when one requirement is clearly subsumed by another.
3. Keep requirements separate when they are merely related but still represent distinct responsibilities.
4. If overlapping requirements conflict, prioritize the most detailed and technically feasible description.

OUTPUT REQUIREMENTS

Return only a valid JSON object. If no redundancies are found, return {"merge_groups": []}.

RESPONSE SCHEMA

```
{"merge_groups": [
  {"reasoning": "...", "merged_name": "...",
   "merged_description": "...", "source_ids":
    [1, 2]}
]}
```

RULES

- Each original id can belong to at most one merge group.
- Each source_ids list must contain at least two ids.
- Keep merged_name and merged_description in the same language as the input.

B. Requirement-Level DAG Construction

The requirement-level DAG prompt asks the model to emit requirement-level functional coordination relations that organize capabilities before component planning.

Requirement-Level DAG Construction Prompt

You are an expert Software Architect. Your task is to analyze project requirements and construct a directed acyclic graph (DAG) that captures *requirement-level functional coordination relations* for architectural planning.

CORE COORDINATION ANALYSIS FRAMEWORK

1. Prerequisite Relation: create edge $A \rightarrow B$ when capability A should be considered before capability B in the requirement space, such as when user-facing behavior in B presupposes concepts, data, or interfaces introduced by A .
2. Support Relation: create edge $A \rightarrow B$ when capability A provides shared behavior, data abstractions, or interface expectations that help define capability B .
3. Shared Functional Concern: create edge $A \rightarrow B$ only when two capabilities should be decomposed, planned, or validated together because they belong to the same functional concern.

CRITICAL FILTERING RULES

- EXCLUDE testing requirements, documentation requirements, and auxiliary development tooling unless they are core to the product.
- EXCLUDE implementation-only relations, such as import, call, inheritance, or file-level containment, because these belong to the component-level DAG.
- EXCLUDE weak topical similarity that does not affect decomposition, planning, or validation.

COORDINATION CREATION BEST PRACTICES

- Create edge $A \rightarrow B$ only when A provides prerequisite or support context for understanding, decomposing, planning, or validating B .
- Ask: "Should these two capabilities be considered together before component-level implementation dependencies are known?" If no, do not create the edge.
- Keep only the most direct coordination relation and avoid transitive edges.
- Use exact requirement names from input; when in doubt, prefer fewer edges.

OUTPUT REQUIREMENTS

1. Output only a valid JSON object, with no explanations or markdown.

2. Format: {"Parent Name": "Child Name"}.
3. If no valid edges exist after filtering, return {}.
4. Both parent and child must be exact requirement names from the input list.

INPUT PLACEHOLDER

Input Project Requirements List:
{requirements_list}

C. Decomposition and Component Planning

The decomposition stage follows a two-stage reasoning-then-labeling process. Repo0 first prompts the LLM to make each high-level requirement self-contained through detailed requirement analysis. It then prompts the LLM to label that analysis into sub-requirements and dependency relations, forming a local requirement-level DAG. The resulting sub-requirements are used to generate a cohesive component plan.

Requirement Analysis Prompt

You are a software architect expert. Analyze the high-level requirement below in as much detail as needed before decomposing it.

Requirement Name: {requirement.name}
Description: {requirement.description}

ANALYSIS REQUIREMENTS

Your analysis should make the requirement self-contained for later decomposition. Cover:

- intended behavior and user-visible functionality
- important inputs, outputs, data contracts, and interface expectations
- constraints, invariants, and error cases
- internal responsibilities that may need separate implementation treatment
- ambiguous or underspecified scope that should be preserved for planning

OUTPUT FORMAT

Do not produce sub-requirements yet. Return only a JSON object:
{"analysis": "detailed self-contained requirement reasoning"}

Sub-Requirement DAG Labeling Prompt

You are tasked with labeling a self-contained requirement analysis into a sub-requirement DAG.

Parent Requirement: {requirement.name}
Original Description: {requirement.description}
Complete Requirement Analysis: {analysis}

INSTRUCTIONS

1. Break the analysis into concrete sub-requirements only where separable requirement-side functionality may need an independent behavioral contract during implementation.
2. Do not target a fixed number of sub-requirements. Use as many or as few as the requirement analysis justifies.
3. Each sub-requirement must describe requirement-side functionality, not packages, files, classes, or implementation components.
4. Avoid documentation-only and testing-only sub-requirements.
5. For each sub-requirement, list dependency indices for earlier sub-requirements that the current sub-requirement logically follows. A dependency from u to v means that v relies on the behavior, output, data contract, or interface assumption established by u .
6. Dependencies must point only to previous sub-requirements using zero-based indices. Use an empty list when the sub-requirement can be understood directly from the enriched requirement description.

OUTPUT FORMAT

Return only a JSON object:

```
{
  "analysis": "brief explanation of how the analysis was segmented",
  "sub_requirements": [
    {
      "name": "short-name-without-parent-prefix",
      "description": "clear requirement-side functionality",
      "depend": [0],
      "rationale": "why this sub-requirement is separated"
    }
  ]
}
```

Architecture Planning Prompt

You are a software architecture expert. Design a unified, cohesive architecture for the following parent requirement and ALL its sub-requirements.

PARENT REQUIREMENT: {parent_name}
Description: {parent_desc}
SUB-REQUIREMENTS TO IMPLEMENT: {sub_reqs_text}
Environment Context:
{environment_feedback}
{dag_context}

CRITICAL INSTRUCTIONS

1. Prefer a compact set of components; introduce additional components only when there is a clear architectural boundary.
2. Each component should serve multiple related sub-requirements; avoid one-to-one mapping between sub-requirements and components.
3. Optimize for the smallest set of high-cohesion, clearly ownable modules, not the longest feature checklist.
4. Merge adjacent responsibilities when they share the same data model, lifecycle, owner, and change together in practice.
5. Do not create separate components for validation, orchestration, export, adapters, helpers, or metadata unless they have clear standalone reuse value.
6. Split components only when there is strong evidence of different interfaces, dependencies, runtime constraints, or independent evolution.
7. Preserve domain semantics and avoid overly generic component names.

OUTPUT FORMAT

Return only a JSON object with components, component_count, and rationale. Each component includes name, responsibilities, and serves_subrequirements.

D. Component Dependencies and Structural Actions

The implementation-level dependency prompt infers must-have component dependencies. The refinement prompts then evolve the component-level structure through save, split, revise, merge, add, and delete, while using the requirement DAG as semantic context rather than as the direct action target.

Component Dependency Inference Prompt

You are analyzing implementation dependencies between planned software components.

GOAL

Infer MUST-HAVE implementation dependencies only. An edge source $\rightarrow$ target means the source component cannot be implemented correctly without importing, invoking, consuming, or structurally depending on the target.

IMPORTANT RULES

1. Cross-requirement edges are allowed and expected when implementation reuse is necessary.

2. Same-requirement edges are also valuable, but return them in a separate list.
3. The requirement DAG edges are soft hints, not hard constraints.
4. Prefer omission over weak guesses.
5. Do not invent components. Only use component ids from the input.
6. Good reasons include API consumption, canonical data/model contract usage, result-object dependencies, configuration/schema ownership, orchestration callbacks, or backend service usage.
7. Bad reasons include topical similarity, shared vocabulary, belonging to the same subsystem, or likely future reuse.

RETURN JSON ONLY WITH THIS SCHEMA

```
{
  "cross_requirement_edges": [...],
  "same_requirement_edges": [...],
  "uncertain_edges": [...],
  "summary": {"notes": "optional short note"}
}
INPUT: {prompt_payload}
```

Structural Action Selection Prompt

You are a software project strategist. Given an architecture design, determine the optimal repository actions.

```
Requirement: {requirement_info.name}
Architecture Components: {components_desc}
```

AVAILABLE ACTIONS

save, split, revise, merge, add, and delete.

ACTION SEMANTICS

- Treat save as the default when the current component boundary and responsibilities are already structurally appropriate.
- Treat split as a structural refinement signal first, not just a response to component size.
- Do not use split to create one module per feature; prefer shared module families for nearby features.
- Treat revise as a boundary-preserving revision that updates responsibilities or implementation strategy without creating a new component.
- Use add primarily during initial structure establishment when a sub-requirement still lacks a concretely realized component; later rounds should prefer save, split, revise, merge, or delete when revising existing structure.
- Preserve requirement vocabulary so later planning can retain domain families.
- In multi-round feedback, consider previous feedback for each component and revise actions only when needed.

OUTPUT FORMAT

Return only a JSON array in the form [{ "component": "...", "action": "...", "rationale": "..." }].

New Requirement Integration Prompt

Context

You are a strict Requirements Analyst. Your goal is to integrate a New Requirement into a DAG.

Input Data

```
New Requirement: {new_requirement}
Existing Requirements Candidates:
{existing_reqs_text}
```

Decision Logic (Follow Strictly)

Step 1: Check for duplicates. If the new requirement has exactly the same semantic meaning, tag it as EXISTING.
 Step 2: Check for independence (the "ADD" test). Ask whether the new requirement can exist or be implemented even if the candidate requirement is deleted. Only if it is clearly independent and not a subset of an existing requirement should it be tagged as ADD.
 Step 3: If Step 2 fails, check for a strict relation. Use CHILD, MERGE, SPLIT, or DELETE only when there is a real dependency, refinement,

or conflict.

CRITICAL CONSTRAINTS

- Do not create a relation just because two requirements share keywords.
- Link them only when there is a clear functional dependency or strong design coupling.
- Balance ADD and RELATION; avoid overusing either tag.

OUTPUT FORMAT (JSON ONLY)

```
{
  "tag": "EXISTING|ADD|RELATION",
  "relation_type":
    "CHILD|MERGE|SPLIT|DELETE|null",
  "affected_requirements": ["name_string"],
  "reason": "...",
  "confidence": 0.0
}
```

Requirement-Level DAG Evolution Prompt

You are analyzing how to integrate a new requirement into an existing requirement DAG.

```
New Requirement: {new_requirement}
Existing DAG Summary: {dag_summary}
Existing Requirements Sample:
{existing_requirements}
Context: {context}
```

ANALYZE THE IMPACT AND DECIDE THE BEST REQUIREMENT-INTEGRATION OPERATION

1. create/add: use when the requirement is genuinely new and does not overlap with existing nodes.
2. split: use when the new requirement shows that an existing node should be broken down.
3. merge: use when the new requirement overlaps substantially with existing nodes.
4. delete: use when the new requirement makes existing nodes obsolete.
5. save: use when the new requirement does not require any structural change to the existing nodes.

RETURN ONLY JSON

```
{
  "operation": "create|add|split|merge|delete|save",
  "reason": "...",
  "confidence": 0.0,
  "target": "existing_requirement_name",
  "targets": ["req_a", "req_b"],
  "suggested_parents": ["parent_requirement"]}
}
```

Metric-Guided Component Splitting Rule

Component splitting is triggered by modularity metrics and executed using requirement-graph partition evidence plus LLM responsibility rewriting. A component becomes eligible only when the metric-action stage marks it with `recommended_action = split` because its cohesion is below the split threshold and it serves enough sub-requirements in the current round.

```
Parent requirement: {parent_task}
Source component payload: {source_component}
```

PROCEDURE

1. Read the ordered, deduplicated `source_component.serves_subrequirements` set. If it contains fewer than two items, keep the component unchanged.
2. Construct the induced requirement subgraph over the served sub-requirements and partition it with a minimum-cut objective: prefer child groups with dense internal functional coordination and sparse cross-group coordination.
3. Keep partitions that satisfy the minimum group-size constraint and do not exceed `split_max_output_components`.

4. Provide the partition groups to the LLM as structural evidence. The LLM derives child component names, responsibilities, and served sub-requirement assignments from these groups.
5. Validate that every child has a non-empty name, non-empty responsibilities, and served sub-requirements drawn only from the source component. Accept the split only if at least two valid children remain.
6. Update the alignment relation by assigning each served sub-requirement to the child component that owns it after the split.

OUTPUT RECORD

```
{
  "reason": "short explanation",
  "confidence": 0.0,
  "split_components": [
    {
      "name": "ChildComponentName",
      "responsibilities": [...],
      "serves_subrequirements": [...]}
  ]
}
```

Metric Merge Judge Prompt

You are a strict software architecture merge reviewer. The metric stage has already proposed one merge candidate from component coupling; your task is only to decide whether the candidate is semantically safe.

Input JSON: {metric_merge_candidate}

COUPLING DEFINITION

The candidate score is $\text{Jaccard}(\text{served_subrequirements}) * \text{layering_penalty}$. The Jaccard term measures overlap between the two served-subrequirement sets. The layering penalty discounts pairs whose names suggest complementary layers such as core/API, engine/integration, or core/adaptor.

APPROVAL RULES

1. Treat high coupling as evidence of similar requirement scope, not as proof that merging is safe.
2. Approve only if the smaller component is mostly redundant and both components have the same core responsibility.
3. Reject merges that would collapse a clean API/engine, parser/builder, core/adaptor, or other layered separation.
4. Reject when there is an interface conflict, behavior conflict, or high architectural risk.
5. Do not merge any component whose recommended action is save or revise; those actions mean the boundary should remain stable for the round.

RETURN JSON ONLY

```
{
  "approved": true,
  "same_responsibility": true,
  "interface_conflict": false,
  "behavior_conflict": false,
  "risk": "low",
  "reason": "short reason"
}
```

E. Code Generation and Repair

Repository materialization first plans file layout, then follows a skeleton-test-fill workflow, after which repair prompts address import, syntax, and lint failures.

File Layout Planning Prompt

You are planning file layout for a Python project.

```
Requirement: {requirement_name}
Description: {requirement_description}
Components: {component_list}
```

RULES

1. Output one file path for each component.
2. File paths must be relative and end with .py.

3. Keep paths under the allowed roots when possible.
4. Prefer stable package layout and avoid creating new top-level roots.
5. Preserve domain semantics from the requirement and served sub-requirements.
6. Avoid collapsing unrelated domain families into generic buckets such as core, common, runtime, api, or services unless a domain-specific nested package is present.
7. When a generic package is unavoidable, prefer core/<domain_family>/module.py over core/module.py.
8. Do not create one subpackage per named feature by default; nearby features should share the same stable module family.

RETURN ONLY JSON

```
{
  "plans": [
    {
      "component_name": "ComponentName",
      "file_path": "package/subpkg/module.py"
    }
  ]
}
```

Skeleton Generation Prompt

You are an expert Python engineer. Produce a SKELETON module only (phase one of two).

```
=== PROJECT REQUIREMENT ===
Name: {req_name}
Description: {req_description}
Dependencies: {req_dependencies}
=== ARCHITECTURE DESIGN ===
Rationale: {architecture_rationale}
=== PARENT CONTEXT ===
Parent Node: {parent_node}
=== COMPONENT ===
Name: {component_name}
Responsibilities: {responsibilities}
=== RELATED COMPONENTS === {context_components}
```

RULES FOR SKELETON

1. Include imports, module docstring, and public classes/functions with full signatures and docstrings.
2. Use `raise NotImplementedError("TDD")` only for concrete members; do not use `pass` or `...` as placeholders in public concrete members.
3. Preserve names and shapes implied by responsibilities; this API will be tested in phase two.
4. Reuse names and import paths from implemented components exactly when needed.
5. Every public declaration must be valid Python syntax and use legal Python identifiers.
6. Core algorithms must be designed and implemented inside this generated repository.

RETURN ONLY JSON

```
{
  "file_path": "<path>.py",
  "code": "<full skeleton source>"
}
```

Pytest Generation Prompt

You write pytest for the following skeleton. Repo root will be on PYTHONPATH.

```
Module relative path: {resolved_file_path}
Import module as: {module_q}
Component: {component_name}
Responsibilities: {responsibilities}
Skeleton source: {skeleton_code}
```

RULES

1. Use pytest style.
2. Tests may initially fail because the skeleton is not implemented yet, but do not lock in `NotImplementedError` behavior for concrete APIs.
3. Only expect `NotImplementedError` for explicitly abstract interfaces.

4. Cover the final public API implied by the skeleton and responsibilities, including return shapes, state changes, and integration behavior where possible.
5. `test_file_path` should be under `tests/` and use a stable name.
6. Treat the reviewed skeleton interfaces as the source of truth; do not invent alternate names.

RETURN ONLY JSON

```
{
  "test_file_path": "tests/test_...py",
  "test_code": "<full pytest file source>"
}
```

Full Implementation Prompt

You now FULLY implement the module so that the provided pytest passes.

```
Module file path: {resolved_file_path}
Import root qualification: {module_q}
Reviewed skeleton: {skeleton_code}
Pytest file: {test_code}
Implemented components context:
{implemented_components_context}
```

RULES

1. Return complete module source in key code only.
2. Remove every concrete placeholder such as `NotImplementedError("TDD")`, `pass`, or `...`
3. Implement according to the reviewed skeleton's public API, docstrings, and stated semantics.
4. Match imports and names from the skeleton exactly.
5. Match import paths and symbol names from implemented components exactly; do not invent new names.
6. If a dependency API is ambiguous, add a small local compatibility wrapper in this file rather than importing a symbol that does not exist.
7. The final module must compile as valid Python.
8. Implement the component's core algorithm locally in this repository or by reusing already generated local modules.

RETURN ONLY JSON

```
{
  "code": "<full implemented source>"
}
```

Lint Patch Repair Prompt

You are fixing Python code to satisfy lint/syntax checks.

```
Attempt: {attempt_no}
Target file path for patch: {virtual_path}
Lint diagnostics: {diagnostics}
Current code: {content}
```

HARD REQUIREMENTS

1. Return only a unified diff patch for this single file.
2. The patch must only touch the target file.
3. Use headers exactly `--- a/<path>` and `+++ b/<path>`.
4. Include valid `@@` hunk lines.
5. Do not include markdown fences or extra text.

Lint Full-File Fallback Prompt

You are fixing Python code to satisfy lint/syntax checks.

```
Attempt: {attempt_no} (full-file fallback mode)
File path: {file_path}
Lint diagnostics: {diagnostics}
Current code: {content}
```

REQUIREMENTS

1. Return only corrected Python code.
2. Keep behavior the same unless required to fix errors.
3. Do not add markdown fences.

Import Postcheck Repair Prompt

Repair a Python import smoke failure for component `{component_name}`.

```
Target module: {module_name}
Planned implementation path: {planned_file_path}
Import failure trace: {failure_trace}
```

GOAL

Make `importlib.import_module(target_module)` succeed.

CONSTRAINTS

1. Fix the import-time or runtime error shown in the trace.
2. Keep file paths unchanged.
3. Preserve existing public APIs and exported symbol names unless the stack trace proves they are wrong.
4. Prefer minimal repairs to top-level initialization, registry or state shape mismatches, and bad imports.
5. Do not remove required behavior just to silence the import error.
6. Do not introduce placeholder code or `NotImplementedError`.